A Comparative Analysis of Mixture-of-Experts and Dense Transformer Architectures: Efficiency, Scalability, and Inference Cost

Overview

This report compares Sparse Mixture-of-Experts (MoE) models and dense Transformer models. While dense models route every token through the entire set of network weights, MoE models conditionally route tokens to a sparse subset of expert feed-forward sub-networks via a gating mechanism. This structural difference fundamentally changes parameter scaling, training compute costs, hardware memory footprints, and runtime inference efficiency.

Comparison

Criterion	Approach A	Approach B
Computational Efficiency (Training)	MoE models achieve significantly faster training throughput for a given compute budget by scaling total parameter count while maintaining a sparse active computational graph per token.	Dense Transformer models execute every layer for every token, resulting in a linear scaling relationship between parameter count, FLOPs per token, and training time.
Scalability	MoE models scale parameter capacity massively without a proportional penalty in training FLOPs, though scaling is bounded by communication bottlenecks across distributed hardware accelerators.	Dense Transformers scale predictability with compute and data following scaling laws, but scaling total capacity requires increasing active FLOPs proportionally for every forward pass.
Inference Cost and Memory Footprint	MoE inference demands high accelerator memory to hold the full parameter set, often leading to GPU memory pressures that require offloading techniques like ExpertFlow, which can reduce GPU	Dense models have straightforward memory requirements matching their active parameter count, making them simpler to cache, quantize, and deploy uniformly on resource-constrained devices.

memory consumption by up to 93.72%.

Strengths & Weaknesses

Approach A

+ Significantly higher parameter capacity without a proportional increase in FLOPs per token
+ Superior downstream task performance at equivalent training compute budgets compared to dense counterparts
+ Enables modular specialization where subsets of experts focus on specific domains or task characteristics
- Massive total parameter memory footprint creates severe memory-bandwidth and hardware-allocation challenges during deployment
- Complex routing overhead and expert load-balancing issues can introduce synchronization bottlenecks in distributed clusters
- Vulnerable to expert starvation and routing collapse if gating functions fail to distribute tokens evenly

Approach B

+ Predictable and straightforward hardware execution paths with minimal control-flow divergence
+ Lower total memory footprint per model instance, easing deployment in single-device or memory-constrained environments
+ Simpler post-training quantization, pruning, and speculative decoding integration without router-weight overhead
- Prohibitive training and active compute costs when scaling model capacity to hundreds of billions of parameters
- Sub-optimal parameter efficiency because all tokens activate the same static weights regardless of complexity
- Requires scaling active FLOPs linearly alongside parameter expansion, limiting performance gains under strict compute budgets

Recommendation

> Use Sparse Mixture-of-Experts models when training compute budgets are constrained relative to the target parameter scale, and when serving infrastructure can support high memory capacities and distributed expert parallelism. Choose dense Transformer models when deployment memory is severely restricted, when low-latency single-device serving is required, or when avoiding the complex communication and routing overhead of MoE systems is paramount.

Sources

ExpertFlow: Efficient Mixture-of-Experts Inference via Predictive Expert Caching and Token Scheduling

Demonstrates how predictive expert caching and scheduling reduce MoE GPU memory usage by up to 93.72% and improve inference throughput by up to 10x.

https://arxiv.org/abs/2410.17954v2

Not All Experts are Equal: Efficient Expert Pruning and Skipping for Mixture-of-Experts Large Language Models

Evaluates expert-level sparsification and pruning techniques to overcome the immense parameter footprint challenges of MoE LLMs during deployment.

https://arxiv.org/abs/2402.14800v2

Convergence Rates for Softmax Gating Mixture of Experts

Analyzes the theoretical convergence properties and parameter estimation behaviors of adaptive softmax gating mechanisms in MoE architectures.

https://arxiv.org/abs/2503.03213v1